Let's Learn C

- By Aditya Varma

Preface

Welcome to Let's Learn C

This book is written to help you learn C as a skill, not as a subject.

C is a language that shows how programs really work. It does not hide details or make assumptions for you. Because of this, learning C builds a strong foundation that makes other languages easier to understand later.

The chapters in this book are arranged carefully. Each concept is introduced only when it is needed, and every idea is supported by simple programs that explain *why* something works, not just *how* to write it.

No prior programming knowledge is assumed. You are expected only to read, practice, and stay curious. This book is meant to be read slowly, like a guide. Type the programs. Modify them. Observe what changes.

Take your time. Ask questions. Make mistakes. That is how real understanding is built.

-Aditya Varma

LICENSE & CREDITS

Credits:

Certain icons, illustrations, and portions of reference content are adapted from publicly available educational platforms, including the Coding X mobile application. Proper credit is given where applicable. All original rights remain with their respective creators, and this material is used solely for educational purposes.

Index

Getting Started with C

C Programming, created by Dennis Ritchie, C is a general-purpose, high-level programming language. It was created at Bell Labs in the early 1970s. One of the first operating systems to be written in C was UNIX. Even today's modern OSs have certain core parts written in C. These include Windows, Mac OS, and even Linux.

C is a perfect choice for learning to code if you are a beginner.

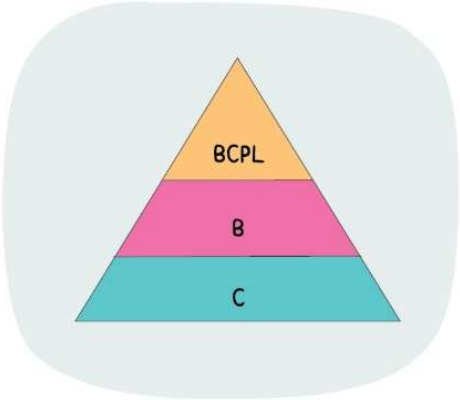
History Of C:

In the late 1960s, a high-level programming language called BCPL was created by Martin Richards, this language aimed to create a readable programming language that can be used to write Operating Systems (OS).

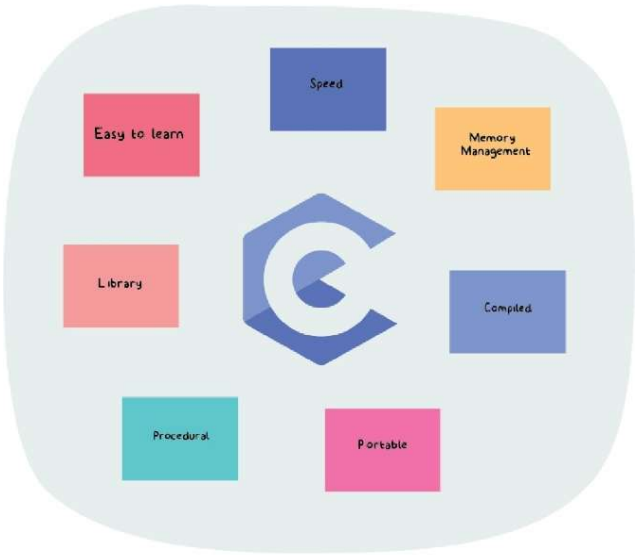
Back in those days, the programs for writing OS were not at all human-readable. This was limiting the adoption of computers for commercial purposes. And hence, this language was invented which was further improved by Ken Thompson and named "B". These languages did solve a lot of problems, but there was still an issue with memory management. Those were the old times and every byte of memory was important.

Dennis Ritchie saw this as a perfect opportunity to improve upon the B language. He tried to provide programmers with lots of memory optimization instructions along with inheriting a bulk of concepts from this existing language, B.

Since it was an improvement over the 'B' language, it was named as C Programming language. Quite convenient, eh?



Features of C:



Let's see a few reasons why you should choose C. Below are some of its amazing features.

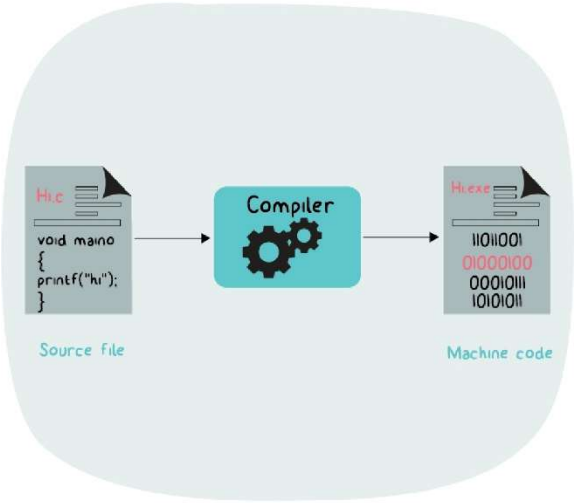
- Easy-to-learn: C has few keywords, a simple structure, and a clearly defined syntax. This makes it easy to catch for beginners
- Speed: The execution time for a C code is not the fastest among all programming languages. But its speed improves as the quantity of data to be processed increases astronomically
- Memory Management: C provides a variety of instructions to handle the memory allocation of its resources. If something can be achieved using lesser resources, C has the means to achieve that
- Broad standard library: C's bulk libraries are very portable and cross-platform compatible with UNIX, Windows, and Macintosh
- Compiled Language: Being a high-level language, its English-like instructions are converted into machine-level code using a software program called Compiler
- Portable: C can run on a wide variety of hardware platforms and has the same interface on all of those
- Procedural Language: Unlike an OOP language that is divided into objects and classes, C is divided into procedures or functions. This is like dividing your code into an exact set of steps to be performed.

How C Works?

C is a compiled language; it uses compiler to execute the program.

But what is a Compiler? It is a computer program that translates code written in one programming language into another language. What we as developers code is very English-based sentences. E.g. - PRINT("SOMETHING"); This instruction needs to be converted into a machine-understandable code.

Also, a developer's code might be distributed in different files (imagine tens of documents). It is the job of the compiler to link all these files and create a final executable object that can run on the system.

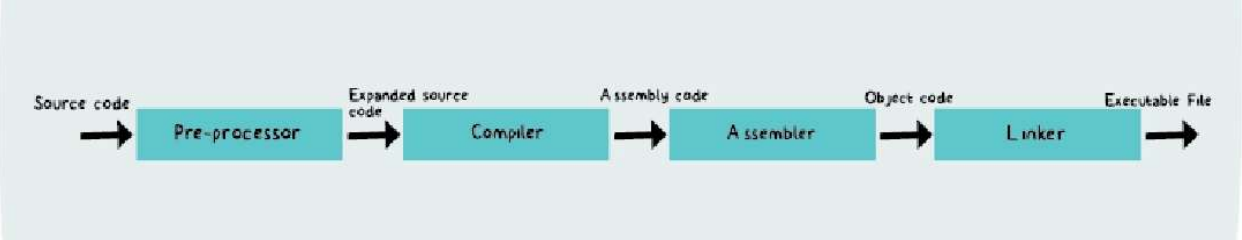


Compiled languages are converted directly into machine code that the processor can execute. As a result, they tend to be faster and more efficient to execute than interpreted languages.

Steps in Compilation:

It can be divided into 4 main tasks and they are performed in the following order,

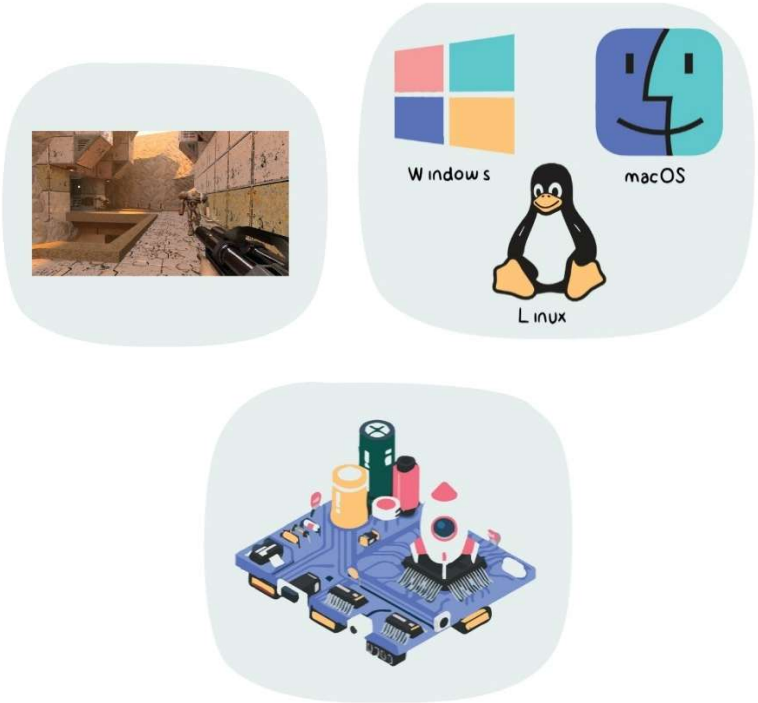
1. Preprocessor: This readies the code for execution by removing any comments, replacing procedure calls with actual code
2. Compilation: It converts pre-processed code into assembly (low-level) language
3. Assembler: This converts assembly language code into machine code, that is a string of 1's and 0's (binary code)
4. Linker: A project will have multiple files containing the source code. The linker will give one final executable file as an output



Where is C used?

Do you remember that we defined C as a general-purpose programming language? This means it is used in most places! And that's the reason why developers love C so much. C is used for,

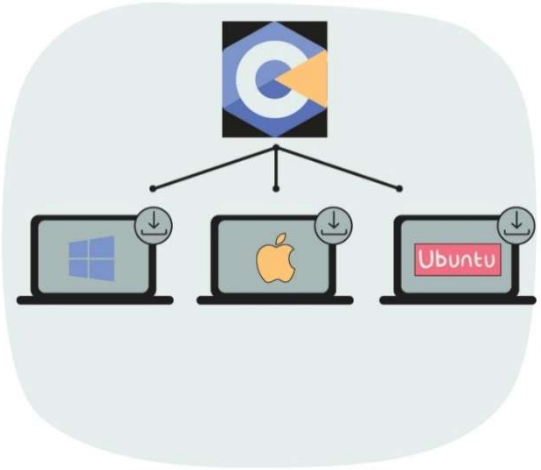
- **Desktop App Development**: C can be used to write any desktop application that you wish to develop. From a music player to photo-editing software, C has been already used. Certain parts of Mozilla Firefox Browser are written using C language.
- **Game Development**: C has a rich set of predefined graphics functions that aid in building video games. Till the early 2000s, C was heavily used for building games. The popular Unity game engine has several parts implemented using C.
- **Embedded Systems**: Due to its feature of memory management, C is the top choice for embedded systems development. It deals with microcontrollers or microprocessors, say, any machine that is controlled via inputs. C is used to create a logic that can control such devices. From rockets flying into space to a tiny smartwatch, C can handle them all!
- **Operating Systems**: We have already seen that C is used in the development of Operating Systems. In fact, it was created for this very purpose. Almost all major operating system's core kernel is written fully or partially using C. Not just operating systems, C is used to build compilers as well. That means new programming languages can also be made in C.



These are just a few of the applications noted here. C's prominent hold is on the embedded systems. That makes it a go-to language if you are thinking of a career in IoT. Of course, your options are not just limited to that.It's a 50 years old language but it surely has a future and rightly proves that - Old is Gold!

Installing C and its IDE:

To work with C on your machine, first, you will need to install the C Compiler in your system. It is unlikely that your system will be shipped with C Compiler.



Almost every C programmer uses the simplest way for getting this by using the GCC compiler. But first, would you like to know what GCC is? GCC stands for GNU Compiler Collection, it is an open-source compiler that supports the execution of multiple programming languages like C, C++, Fortran, Golang, and more.

GCC has been ported to multiple platforms and hence it helps in building C programs on a wide variety of systems. Currently, GCC's appropriate version for multiple platforms is available widely and for free.

But we are going to use something more efficient i.e., “Codeblocks”.

Installing Code::Blocks with MinGW:

Before we start writing C programs, we need a place to write and run them. For this, we will use Code::Blocks, which comes with a built-in C compiler called MinGW.
Follow the steps carefully.

Step 1: Download Code::Blocks

- Open your browser
- Search for: Code::Blocks official website
- Open the website:
  👉 Code::Blocks
- Go to the Downloads section
- Click on “Download the binary release”

Step 2: Choose the Correct Version

You will see multiple options. Choose the one that includes MinGW.

- Look for a file named similar to:

  codeblocks-20.03mingw-setup.exe

⚠️ Important:

- Always choose the version with "mingw" in the name
- This version already contains the C compiler
- No need to install anything separately

Step 3: Install Code::Blocks

- Double-click the downloaded .exe file
- Click Next → I Agree → Next
- Keep default settings
- Click Install

  Once installation completes:
- Click Finish

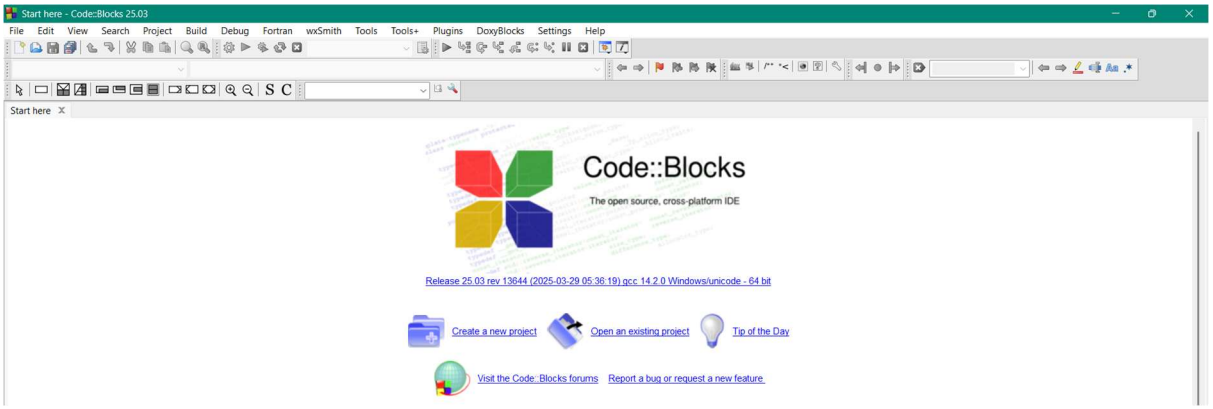
Step 4: First Launch Setup

- Open Code::Blocks
- It may ask to select a compiler

If prompted:

- Choose: GNU GCC Compiler
- Click Set as default
- Click OK

The setup is done and screen will look like this:



Creating files in Code::Blocks:

Step 1: Create Your First Project

- Click on Create a new project
- Select Console Application
- Click Go → Next
- Choose language: C
- Click Next

Step 2: Set Project Details

- Enter a Project Title (e.g., MyFirstProgram)
- Choose a folder location
- Click Next → Finish

Step 3: Write Your First Program
You will see a default file (main.c).

- Inside it, you can write your first program
- Or modify the existing code

Step 4: Build and Run the Program

- Click on Build and Run (green play button)
  OR press F9
- Output will appear in a console window

At this stage, do not worry about:

- How the compiler works internally
- Complex settings
- Advanced configurations

Right now, your goal is simple:
Write → Run → Observe

First C Program & Boilerplate

After setting up the environment, the next step is simple: write something and see it work. Your first program is not about complexity. It is about starting a conversation with the computer.

Let us begin with a small program:

First.c

```c
#include <stdio.h>

int main() {
    printf("Hello, World!");
    return 0;
}
```

When you run this program, the output will be:

```
Hello, World!
Process returned 0 (0x0)   execution time : 0.163 s
Press any key to continue.
```

This is your program speaking back to you.

Understanding the Program

Let us read the program like we read a sentence.

The line `#include <stdio.h>` is written at the top. This line tells the program to bring in a set of tools before starting. These tools help in performing input and output operations. Since we are using `printf()` to display something on the screen, this line becomes necessary.

The next line, `int main()`, is where execution begins. Every C program starts from `main()`. When the program runs, the computer looks for this function and starts executing from here.

The curly braces `{ }` define a block. Everything written inside these braces belongs to the `main()` function. It is simply a way of grouping instructions together.

Inside this block, we have the line: `printf("Hello, World!");`
This is the instruction that produces output. `printf()` is used to display text on the screen, and whatever is written inside the quotes gets printed.

Finally, we see: `return 0;`
This marks the end of the program. It tells the system that the program has finished executing successfully.

Trying It Yourself:

Once you run the program, try changing the message:

```
printf("I am learning C.");
```

Or:

```
printf("This is my first program.");
```

Run it again and observe the output.
This simple action changing something and observing the result is where real learning begins.

Boilerplate Code

If you look at the program again, you will notice something interesting. Some parts of the program are not changing. No matter what you print, certain lines remain the same.

This repeated structure is called **boilerplate code**.

Boilerplate code is the basic structure required for a C program to run. It is not specific to one program. Instead, it acts as a standard starting point.

A simple version of boilerplate looks like this:

```c
#include <stdio.h>

int main() {
    // your code will go here
    return 0;
}
```

This structure will appear in almost every C program you write.

Understanding Boilerplate Naturally

Instead of memorizing it, try to understand its role.

- The `#include` line brings in required tools.
- The `main()` function defines where execution starts.
- The curly braces hold the instructions.
- The `return 0` ends the program properly.

Everything you write will go inside this structure.

Comments & Escape Sequence

Comments in C:

When we write programs, we are not just writing for the computer. We are also writing for ourselves and for others who may read the code later. A computer only understands instructions. But humans need explanations.

This is where **comments** come in. Comments are lines in a program that are **ignored by the compiler**. They do not affect how the program runs. Instead, they help us **understand the code better**. You can think of comments as small notes written inside a program. They explain what a particular part of the code is doing, or why it was written that way.

As programs grow, they become harder to read. Without comments, even your own code can feel confusing after some time.

Comments help in:

- Explaining the purpose of the program
- Describing what a section of code does
- Making the code easier to read and maintain
- Helping others understand your logic

Even a simple program becomes clearer when comments are used properly.

Types of Comments in C:

- Single-Line Comments

  A single-line comment starts with //. Everything written after // on that line is treated as a comment. This type of comment is useful for short explanations.

  Example:

```
// This is a single-line comment
```

- Multi-Line Comments

  A multi-line comment starts with /* and ends with */. Everything written between these symbols is treated as a comment. This is useful when you want to write longer descriptions.

  Example:

```
/*
This is a multi-line comment.
It can span across multiple lines.
*/
```

Lesson Program: Comments in C

Source Code: Comments.c

```c
#include <stdio.h>

int main() {

    // Printing a welcome message
    printf("Welcome to C Programming!\n");

    // Printing another line
    printf("This program shows comments usage.\n");

    /*
    The following lines perform simple addition
    and display the result
    */

    int a = 10;   // First number
    int b = 20;   // Second number
    int sum = a + b; // Adding numbers

    printf("Sum = %d\n", sum);

    return 0; // Program ends
}
```

Escape Sequence:

When we print something on the screen, the output appears exactly as written.But sometimes, we want more control.

We may want:

- text on a new line
- proper spacing
- special characters like quotes

This is where **escape sequences** are used. Escape sequences are **special character combinations** that begin with a backslash \. They tell the computer to perform a specific formatting action instead of printing normal text.

Without escape sequences, everything would print in a single continuous line, making output hard to read. This makes the output more structured and readable.

List of Escape Sequences in C

Here are the most useful escape sequences:

- \n → New line
- \t → Horizontal tab (space gap)
- \b → Backspace (removes previous character)
- \r → Carriage return (moves cursor to beginning of line)
- \\ → Prints a backslash \
- \" → Prints double quotes "
- \' → Prints single quote '
- \0 → Null character (end of string)

Lesson Program: Escape Sequence in C

Source Code: EscapeSequence.c

```c
#include <stdio.h>

int main() {
    printf("Demonstrating Escape Sequences in C\n\n");

    printf("1. Newline -> Hello\\nWorld:\n");
    printf("Hello\nWorld\n\n");

    printf("2. Tab -> Hello\\tWorld:\n");
    printf("Hello\t World\n\n");

    printf("3. Backspace -> Helloo\\bWorld:\n");
    printf("Helloo\b World\n\n");

    printf("4. Carriage Return -> Hello World\\rHi:\n");
    printf("Hello World\rHioo\n\n");

    printf("5. Backslash -> \\\\:\n");
    printf("\\\n\n");

    printf("6. Single Quote -> \\\':\n");
    printf("\'\n\n");

    printf("7. Double Quote -> \\\" :\n");
    printf("\"\n\n");

    printf("8. Alert -> \\a:\n");
    printf("\a\n"); // may beep if supported

    return 0;
}
```

Try it yourself:

1. Write a C program using printf() that produces the following formatted output using escape sequences only (don't add extra spaces manually):

```
***** Escape Sequences Practice *****

Hello, "Students"!
Welcome to C Programming.

Here is a Poem:

        Twinkle, Twinkle, little star,
        How I wonder what you are!
                Up above the world so high,
                Like a diamond in the sky.\n

End of Poem.
```

2. Write a C program using printf() that produces the following output in a **table format** (use only escape sequences, not spaces):

```
Name            Age     City
----            ---     ----
Rahul           21      Mumbai
Anita           19      Delhi
John            22      New York
```

3. Write a C program to print the following shape using escape sequences:

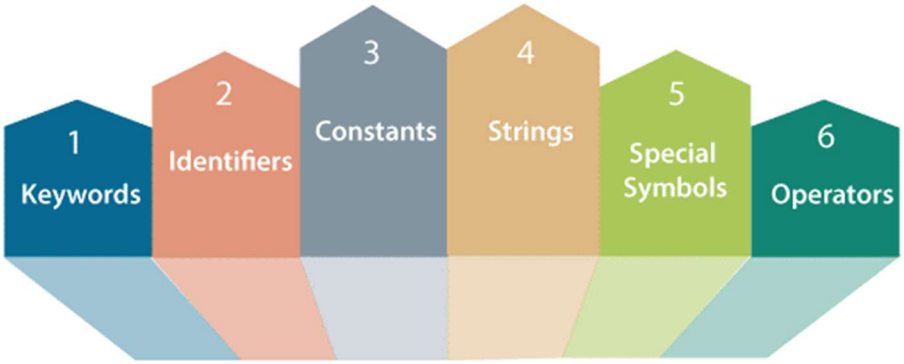
```
          *
         * *
        *   *
       *     *
      *********

Process returned 0 (0x0)
Press any key to continue.
```

The Language Building Blocks: Tokens

In C programming, a token is the smallest individual unit of a program that is meaningful to the compiler. Tokens are the fundamental building blocks used to construct C programs. The C compiler analyses a program by breaking it down into these tokens to understand its structure and functionality during the compilation process.

These are primary types of tokens in C:

- Keywords
- Identifiers
- Constants & Variables
- Operators
- Data types
- String literal



Character Set:

The Character Set of any programming language indicates the different characters the program can contain. The C language identifies a character set that comprises English alphabets upper and lowercase (A to Z, as well as a to z), digits 0 to 9, and certain other symbols with a special meaning attached to them.

- Uppercase: A to Z
- Lowercase: a to z
- Digits: 0 to 9
- Special characters: ! " # $ % & ' ( ) * + - . : , ; ` ~ = < > { } [ ] ^ _ \ /
- Other special characters: Blank space, tab, etc. (Compiler ignores white spaces unless they are part of string)

Keywords & Identifiers:

Every C word is classified as either a keyword or an identifier.

C Keywords:

In C, a predefined sequence of alphabets is called a keyword. Compared to human languages, programming languages have fewer keywords. To start with, C had 32 keywords, later on, few more were added in subsequent revisions of C standards. All keywords are in lowercase.

List of Keywords:

break	case	char	const	auto	short	struct	switch
double	int	else	enum	float	continue	sizeof	default
extern	for	do	goto	if	typedef	union	void
static	signed	long	register	return	unsigned	volatile	while

C Identifiers:

In C, names given to variables, functions, arrays, and other user-defined items are called identifiers. Compared to keywords, identifiers are created by the programmer to uniquely represent data and functions in a program. Unlike keywords, identifiers are not predefined; instead, they are chosen by the programmer while following certain rules.

Rules for Defining Identifiers in C,

1. Identifiers must begin with a **letter (a-z, A-Z)** or an **underscore (_)**.
2. After the first character, they can contain **letters, digits (0-9), or underscores**.
3. Identifiers **cannot be a keyword**.
4. Identifiers are **case-sensitive** (e.g., Age and age are different).
5. No special characters (like @, #, $, %) or spaces are allowed.
6. Identifiers can be of any length, but only the first **31 characters** are significant in older C standards (modern compilers may allow more).

✔ Example Valid Identifiers:

- sum, totalMarks, _count, Age1

❌ Example Invalid Identifiers:

- int (keyword), 2ndValue (starts with digit), my-name (special character)

Lesson Program: Keywords and Identifiers in C
Source Code: KeywordsAndIdentifiers.c

```c
#include <stdio.h>   // include → keyword, stdio.h → header file

int main() {   // int → keyword, main → identifier

    int age = 23;            // int → keyword, age → identifier
    float height = 5.9;      // float → keyword, height → identifier
    char grade = 'A';        // char → keyword, grade → identifier

    printf("Age: %d\n", age);       // printf → identifier
    printf("Height: %.1f\n", height);
    printf("Grade: %c\n", grade);

    return 0;   // return → keyword

}
```

Constants & Variables:

Every C program works with **data**. Data in C can be classified as either a **constant** or a **variable**.

C Constant:

In C, a constant is a fixed value that does not change during the execution of a program. Constants are used to represent values that remain the same throughout.
Constants can be of different types:

- Integer constants → 10, -25, 0
- Floating-point constants → 3.14, -0.001, 2.5e3
- Character constants → 'A', 'z', '1'
- String constants → "Hello", "C Programming"

Symbolic constants: Data is stored as a constant and cannot be changed. Defined using #define or const keyword

C Variable:

In C, a variable is a named location in memory that stores a value which can change during program execution. Variables must be declared before use, and their type defines the kind of data they can hold.
Example: int age = 20; // integer variable
float height = 5.9; // float variable
char grade = 'A'; // character variable
Rules for Defining Variables in C are the same as rules for Identifiers.

Lesson Program: Constants and Variables in C
Source Code: ConstantsAndVariables.c

```c
#include <stdio.h>
#define PI 3.14   // symbolic constant

int main() {
    const int days = 7;      // constant
    int age = 23;            // variable
    float height = 5.9;      // variable
    char grade = 'A';        // variable

    printf("PI = %.2f\n", PI);
    printf("Days in a week = %d\n", days);
    printf("Age = %d, Height = %.1f, Grade = %c\n", age, height, grade);

    return 0;
}
```

Variables And Data Types

Until now, we have written programs that display output. But real programs do more than just print messages. They **store values**, **use them**, and sometimes **change them over time**.

Think about this simple situation:

- A student has marks
- A shop has prices
- A program calculates a result

All of these require the program to **remember values**. A computer does not "remember" things the way we do. It stores everything inside its memory. This is where variables and data types comes.

Variables in C:

A variable is a **place in memory** where a value is stored. When a program runs, the computer uses its memory to hold data. Instead of remembering memory locations directly, we give them **names**. These names are called **variables**.

So, in simple terms: A variable is a named storage location used to hold data.

Each variable:

- stores a value
- has a type (what kind of data it holds)
- is stored somewhere in memory (with an address)

Different types of data require different amounts of memory and support different kinds of operations.

Variables and Memory:

You can imagine memory as a collection of boxes:

- Each box has a **name** → variable
- Each box stores a **value**
- Each box has a **type** → defines what can be stored

For example:

int age = 21;

- int → type (integer)
- age → variable name
- 21 → value stored

Naming Rules (Same as Identifiers in C):

Variable names must follow certain rules:

- Must start with a letter (a–z / A–Z) or underscore _
- Can contain letters, digits, and underscore
- Cannot start with a number
- Cannot be a keyword (int, return, etc.)
- No spaces or special symbols (@ # $ % -)
- Case-sensitive (score and Score are different)
- Should be meaningful (totalMarks, count, price)

Examples of Valid Names:

```
int total;
int _index;
int value2;
```

Declaring a Variable:

Declaration means telling the compiler: "I want to create a variable of this type with this name"

```
int age;
float salary;
char grade;
```

At this stage:

- Memory is reserved
- But no value is stored yet

Assigning a Value:

Assignment means giving a value to an already declared variable.

```
age = 21;
salary = 55000.0f;
grade = 'A';
```

This updates the value stored in memory.

Initializing a Variable:

Initialization means declaring and assigning a value at the same time.

```
int year = 2025;
float pi = 3.14159f;
char letter = 'C';
```

This is the most common and recommended way.

Lesson Program: Variables in C

Source Code: Variables.c

```c
#include <stdio.h>

int main() {

    // Declaration
    int age;
    float salary;
    char grade;

    // Assignment
    age = 21;
    salary = 55000.0f;
    grade = 'A';

    // Initialization (declaration + value together)
    int year = 2025;
    float pi = 3.14f;
    char section = 'B';

    // Using and printing variables
    printf("Age: %d\n", age);
    printf("Salary: %.2f\n", salary);
    printf("Grade: %c\n", grade);

    printf("Year: %d\n", year);
    printf("Pi: %.2f\n", pi);
    printf("Section: %c\n", section);

    return 0;
}
```

Data Types in C:

In C, data types define the type of data a variable can store, the memory it occupies, and the range of values it can hold.

C data types are classified into:

- Primary (Fundamental) Data Types
- Derived Data Types (arrays, pointers, structures, unions)
- User-defined Data Types (typedef, enum, struct, union)

Primary Data Types in C:

Primary (fundamental) data types are the basic built-in types supported directly by the compiler. They include integers, floating-point numbers, characters, and void.

Primary Data Types with Size & Range:

Data Type	Size (bytes)	Range	Format Specifier
short int	2	-32,768 to 32,767	%hd
unsigned short int	2	0 to 65,535	%hu
unsigned int	4	0 to 4,294,967,295	%u
int	4	-2,147,483,648 to 2,147,483,647	%d
long int	4	-2,147,483,648 to 2,147,483,647	%ld
unsigned long int	4	0 to 4,294,967,295	%lu
long long int	8	-(2^63) to (2^63)-1	%lld
unsigned long long int	8	0 to 18,446,744,073,709,551,615	%llu
signed char	1	-128 to 127	%c
unsigned char	1	0 to 255	%c
float	4	1.2E-38 to 3.4E+38	%f
double	8	1.7E-308 to 1.7E+308	%lf
long double	16	3.4E-4932 to 1.1E+4932	%Lf

Note: Size and ranges may vary slightly depending on compiler and architecture (16-bit, 32-bit, 64-bit).

Lesson Program: Data Types in C

Source Code: DataTypes.c

```c
#include <stdio.h>

int main() {
    char letter = 'A';                    // char
    unsigned char u_char = 250;           // unsigned char
    int age = 23;                         // int
    unsigned int u_age = 40000;           // unsigned int
    short int s_num = -30000;             // short int
    unsigned short int us_num = 60000;    // unsigned short int
    long int population = 2000000000;     // long int
    unsigned long int u_population = 4000000000; // unsigned long int
    float pi = 3.14159;                   // float
    double big = 123456.789012345;        // double
    long double huge = 3.14159265358979323 8L; // long double

    printf("char: %c\n", letter);
    printf("unsigned char: %u\n", u_char);
    printf("int: %d\n", age);
    printf("unsigned int: %u\n", u_age);
    printf("short int: %d\n", s_num);
    printf("unsigned short int: %u\n", us_num);
    printf("long int: %ld\n", population);
    printf("unsigned long int: %lu\n", u_population);
    printf("float: %.5f\n", pi);
    printf("double: %.12lf\n", big);
    printf("long double: %.18Lf\n", huge);

    return 0;
}
```

Printing Variables in C

Once a variable stores a value, the next step is to **see and use that value**. In C, this is done using the printf() function. But printing is not just about displaying values. The computer must know **what kind of data** it is printing.

This is where **format specifiers** come in.

Format Specifiers:

A format specifier is a placeholder inside printf() that tells the program: "What type of value should be printed here" Each data type has its own format specifier.

For example: printf("%d", n);

Here:

- %d → expects an integer
- n → value to be printed

Common Format Specifiers

Data Type	Specifier	Example
int, short	%d	printf("%d", n);
unsigned int	%u	printf("%u", u);
long int	%ld	printf("%ld", L);
long long int	%lld	printf("%lld", LL);
float	%f	printf("%f", f);
double	%lf	printf("%lf", d);
char	%c	printf("%c", ch);
string	%s	printf("%s", name);
size_t	%zu	printf("%zu", sizeof(int));
pointer	%p	printf("%p", ptr);

Width and Precision

You can control how values appear on the screen.

- %.2f → prints only 2 decimal places
- %5d → prints number with width 5
- %-5d → left-align within width

These help make output **clean and structured**.

Printing Expressions

printf() can also evaluate expressions directly.

```
printf("2 + 3 = %d\n", 2 + 3);
```

The expression is calculated first, then printed.

This means you can print:

- calculations
- formulas
- results directly

Lesson Program: Format Specifiers in C

Source Code: FormatSpecifiers.c

```c
#include <stdio.h>
#include <stddef.h>

int main(void) {

    int n = 123;
    unsigned int u = 40000u;
    long int L = 2000000000L;
    long long int LL = 900000000000LL;
    float f = 3.14159f;
    double d = 2.718281828;
    char ch = 'C';
    char name[] = "Ada";

    // Printing different data types
    printf("int: %d\n", n);
    printf("unsigned: %u\n", u);
    printf("long: %ld\n", L);
    printf("long long: %lld\n", LL);
    printf("float: %f\n", f);
    printf("double: %lf\n", d);
    printf("char: %c\n", ch);
    printf("string: %s\n", name);

    // Width and precision
    printf("pi (2 decimals): %.2f\n", f);
    printf("n right-aligned width 6: |%6d|\n", n);
    printf("n left-aligned width 6 : |%-6d|\n", n);

    // Printing expressions
    printf("2 + 3 = %d\n", 2 + 3);
    printf("Area 7 x 5 = %d\n", 7 * 5);

    // Sizes and addresses
    printf("sizeof(int) = %zu bytes\n", sizeof(int));
    printf("address of n = %p\n", (void*)&n);

    return 0;
}
```

Operators in C